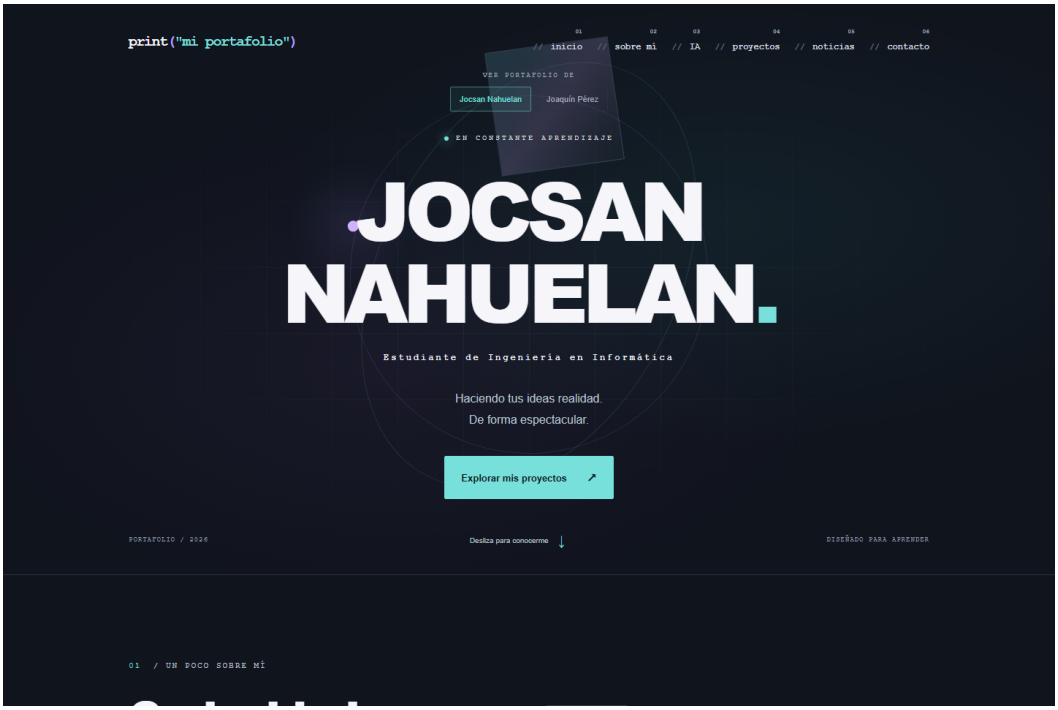
INFORME TÉCNICO

Portafolio personal compartido

Evaluación Formativa 2 · React, Atomic Design y pruebas frontend

Jocsan Nahuelan y Joaquín Pérez · 23 de octubre de 2026



Vista de inicio del perfil de Jocsan, capturada en Chromium desde la versión local.

Correcciones informadas por el equipo: la evaluación se realizará en grupo y se utilizará Vitest con Chromium en lugar de Jasmine y Karma, nombrados en el PDF recibido.

1. Qué hace el portafolio

La página muestra una presentación inicial y deja elegir entre Jocsan y Joaquín. Según el perfil activo, cambia la biografía, la fotografía, las tecnologías, los proyectos, las noticias y los enlaces sociales. La sección de inteligencia artificial aparece solo cuando el JSON del perfil contiene información para ella. El formulario comprueba los datos en el navegador y presenta una confirmación local; no envía correos.

Tecnologías

Tecnología	Uso
React y JavaScript	Componentes reutilizables, props y estado.
Vite	Servidor local y compilación para producción.
Bootstrap y CSS	Cuadrícula adaptable y estilos propios.
JSON	Datos independientes de ambos perfiles, proyectos y noticias.
Vitest, Testing Library y JSDOM	Pruebas unitarias del DOM, eventos y validaciones.
Vitest Browser Mode, Playwright y Chromium	Pruebas de interacción seleccionadas en un navegador real.

Datos y estado

```
import jocsan from './jocsan.json'
import compañero from './compañero.json'

const perfiles = [jocsan, compañero]
```

`perfiles.js` importa cada JSON y crea una lista ordenada. Vite incorpora esos datos al compilar; esta versión no consulta una API. Cada JSON contiene nombre, fotografía, biografía, redes, habilidades, proyectos y noticias. `ia: null` deja oculto el bloque de IA.

```
const [perfilActual, setPerfilActual] = useState(0)
const perfil = perfiles[perfilActual]

<ProjectsSection proyectos={perfil.proyectos} />
```

`useState(0)` recuerda que Jocsan está seleccionado. El botón del selector llama a `setPerfilActual(indice)`. React vuelve a dibujar las secciones con el objeto `perfil` actual. El índice es el estado; el objeto de perfil se deriva de ese índice.

2. Atomic Design y conexión de componentes

Atomic Design ordena la interfaz desde piezas pequeñas hasta la página completa. En React, una llamada ocurre cuando se escribe un componente como etiqueta dentro del JSX de otro. Quien lo usa puede enviarle información por props.

Nivel	Ejemplos	Qué hace
Átomos	Button, Heading, Image, Input, Label, Link, TextArea	Elementos pequeños y reutilizables.
Moléculas	FormField, NavigationItem, NewsCard, ProjectCard	Unen átomos para una tarea concreta.
Organismos	Navbar, AboutMe, ProjectsSection, NewsSection, ContactForm, Footer	Arman secciones completas.
Plantilla	PortfolioTemplate	Ubica navegación, contenido y pie.
Página	PortfolioPage	Elige el perfil y conecta las secciones.

La página conecta los niveles

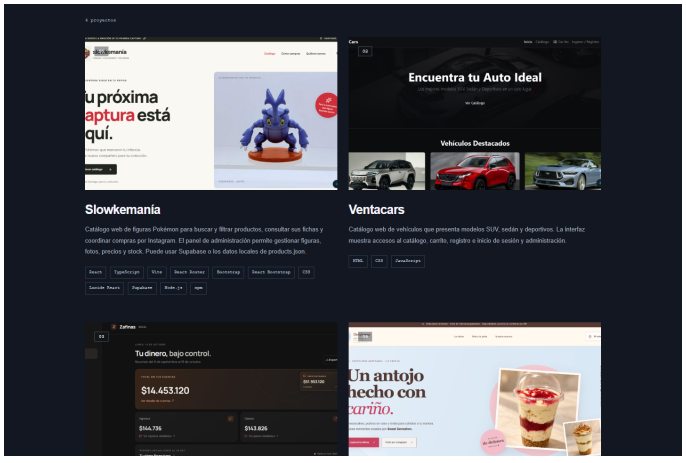
```
<PortfolioTemplate key={perfil.id} perfil={perfil}>
  <Presentation perfil={perfil} perfiles={perfiles} ... />
  <AboutMe perfil={perfil} />
  <ProjectsSection proyectos={perfil.proyectos} />
  <NewsSection noticias={perfil.noticias} ... />
  <ContactForm />
</PortfolioTemplate>
```

`PortfolioPage` coordina. `PortfolioTemplate` agrega Navbar, el enlace para saltar al contenido y Footer; `children` son las secciones que se colocan dentro de `<main>`. `key={perfil.id}` reconstruye la plantilla al cambiar de persona y reinicia el menú, el filtro y el formulario.

Una molécula dentro de un organismo

```
{proyectosVisibles.map((proyecto) => (
  <ProjectCard key={proyecto.id} proyecto={proyecto} />
))}
```

`ProjectsSection` recorre la lista y llama a `ProjectCard` por cada proyecto. La tarjeta recibe un objeto y muestra título, descripción, imagen, tecnologías y enlace cuando están disponibles. La prop `key` identifica cada tarjeta en la lista.



La sección muestra las cuatro tarjetas cargadas desde el JSON de Jocsan.

3. Interacciones principales

Proyectos y filtros

```
const [filtro, setFiltro] = useState('Todos')
const proyectosVisibles = filtro === 'Todos'
  ? proyectos
  : proyectos.filter((proyecto) => proyecto.categoria === filtro)
```

`filtro` recuerda la categoría seleccionada. Los JSON actuales tienen la categoría vacía, así que se oculta el selector. Cuando se agreguen categorías, los botones filtrarán la lista y actualizarán el contador.

Noticias desde JSON

```
const noticiasVisibles = noticias.filter(
  (noticia) => noticia.categoria === categoria
)
```

La página crea dos secciones: noticias personales y notas de aprendizaje. `NewsSection` conserva solo las noticias de la categoría recibida y crea una tarjeta para cada una, con título, fecha y contenido.

Formulario de contacto

```
function manejarEnvio(evento) {
  evento.preventDefault()
  if (!nombre.trim() || !correo.trim() || !mensaje.trim()) {
    setError('Completa tu nombre, correo y mensaje.')
    return
  }
  if (!correoValido(correo)) {
    setError('Escribe un correo electrónico válido.')
    return
  }
  setConfirmacion('Mensaje validado localmente')
}
```

Cada campo guarda su valor con `useState`. `preventDefault()` evita recargar la página. El formulario rechaza campos vacíos y un correo con formato incorrecto; si los datos pasan, muestra una confirmación y limpia los campos. No envía el mensaje.

Navegación y adaptación

`Navbar` recorre una lista y crea un `NavigationItem` por destino. `menuAbierto` controla el menú móvil y actualiza `aria-expanded`. Bootstrap aporta clases como `container`, `row`, `col-md-6` y `col-lg-6`; los estilos propios definen la identidad visual.

Accesibilidad y rendimiento

Hay textos alternativos de imágenes, enlace para saltar al contenido, nombres accesibles para navegación y selector de perfil, y roles `alert` y `status` para mensajes. Falta documentar una revisión con teclado, lector de pantalla, Axe o Lighthouse. Vite minifica los recursos: CSS 244.55 kB (34.66 kB gzip) y JavaScript 241.00 kB (75.21 kB gzip). No se registró una medición de velocidad ni una optimización de imágenes.

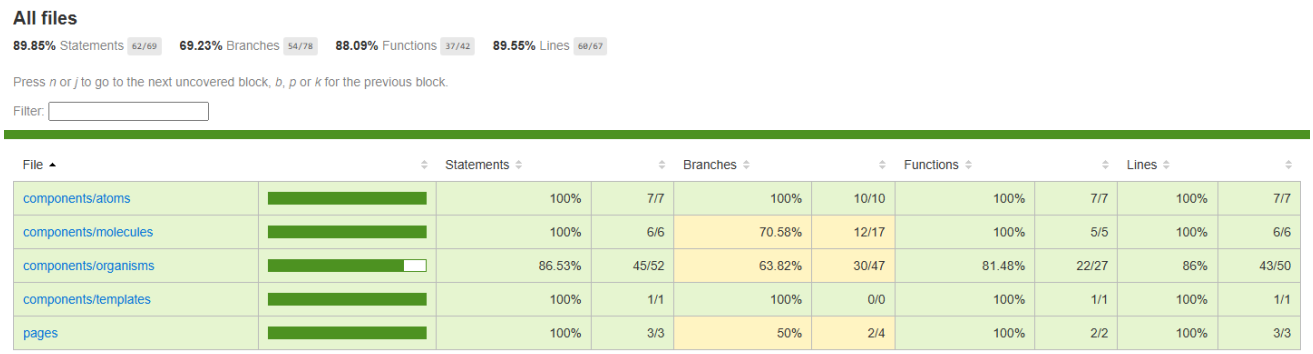
4. Pruebas y cobertura

El PDF nombra Jasmine y Karma; el equipo informó una corrección del docente para usar Vitest y Chromium. Las pruebas unitarias usan JSDOM. Browser Mode abre la página con Playwright en Chromium.

Ejecución	Casos verificados	Resultado
`npm run test -- --run`	Fixture, formulario válido, correo inválido, cambio de perfil	4/4 aprobadas
`npm run test:browser -- --run`	Fixture, formulario y cambio de perfil en Chromium	3/3 aprobadas
`npm run test:coverage`	Cobertura V8 de componentes y páginas	Informe generado
`npm run build`	Compilación de producción	Aprobada

Resultados de cobertura

Medida	Resultado	Significado
Statements	89.85% · 62/69	Instrucciones ejecutadas
Branches	69.23% · 54/78	Ramas condicionales recorridas
Functions	88.09% · 37/42	Funciones llamadas
Lines	89.55% · 60/67	Líneas ejecutadas



Captura del informe HTML de Vitest y V8.

La configuración incluye `src/components` y `src/pages`; no incluye `App.jsx`, los JSON ni la configuración. Las fixtures son datos de ejemplo, no respuestas de un backend. La cobertura alcanzada no demuestra que todos los recorridos estén probados: falta probar menú móvil, filtros, noticias, perfiles incompletos y teclado. `npm run lint` detectó 22 imports `React` sin uso y una advertencia del archivo generado de cobertura.

Conclusión

El portafolio integra dos perfiles en una aplicación React y organiza la interfaz con Atomic Design. Los datos de cada persona se mantienen en archivos JSON; la página selecciona el perfil activo, pasa sus datos mediante props y actualiza la vista con el estado de React.

Las pruebas unitarias y las pruebas de interacción en Chromium aprobaron en sus ejecuciones registradas, y la compilación de producción terminó correctamente. El informe de cobertura describe el alcance configurado para componentes y páginas. En conjunto, el proyecto permite explicar cómo se conectan los datos, los componentes y las interacciones de la aplicación.